

Факультет физико-математических и естественных наук

Кафедра прикладной информатики и теории вероятностей

Отчёт по лабораторной работе №14

Программирование в командном процессоре ОС UNIX. Расширенное программирование

Студент: ТУЙИШИМЕ Тьерри

Группа: НКАбд-05-25

Содержание

1	Цель работы	2
2	Задание	2
3	Выполнение лабораторной работы.....	2
3.1	командный файл, реализующий упрощённый механизм семафоров	2
3.2	Реализовать команду <code>map</code> с помощью командного файла.....	3
3.3	написать командный файл, генерирующий случайную последовательность букв латинского алфавита.	5
4	Выводы	5
5	Ответы на контрольные вопросы.....	5
	Список литературы	7

1 Цель работы

Изучить основы программирования в оболочке ОС UNIX. Научиться писать более сложные командные файлы с использованием логических управляющих конструкций и циклов.

2 Задание

1. Написать командный файл, реализующий упрощённый механизм семафоров.
2. Реализовать команду `map` с помощью командного файла.
3. Используя встроенную переменную `$RANDOM`, написать командный файл, генерирующий случайную последовательность букв латинского алфавита.

3 Выполнение лабораторной работы

3.1 командный файл, реализующий упрощённый механизм семафоров

Чтобы создать данный командный файл, я создала новый файл и написала в нем некоторый скрипт. Он устанавливает переменную `lockfile` для пути к файлу блокировки, открывает файл для записи и назначает ему дескриптор файла. Далее входит в цикл, который выполняется, пока файл блокировки существует. Пытается получить эксклюзивную блокировку для файла. Если это удастся, выводит `"file locked"`, ждет 5 секунд а затем выводит `"file unlocked"`:

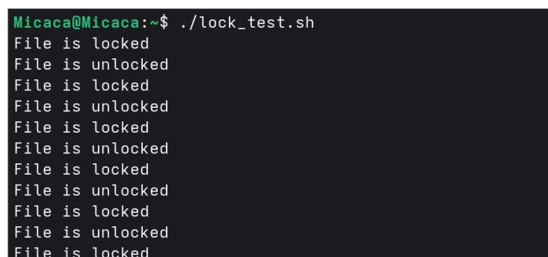


```
1 lockfile="./lock.file"
2 exec {fn}>$lockfile
3 while test -f "$lockfile"
4 do
5     if flock -n ${fn}
6     then
7         echo "File is locked"
8         sleep 5
9         echo "File is unlocked"
10        flock -u ${fn}
11    else
12        echo "File is locked"
13        sleep 5
14    fi
15 done
```

Рис. 1: упрощённый механизм семафоров (код)

```
lockfile="./lock.file"
exec {fn}>$lockfile
```

```
while test -f "$lockfile"
do
if flock -n ${fn}
then
    echo "File is locked"
    sleep 5
    echo "File is unlocked"
    flock -u ${fn}
else
    echo "File is locked"
    sleep 5
fi
done
```



```
Micaca@Micaca:~$ ./lock_test.sh
File is locked
File is unlocked
File is locked
File is unlocked
File is locked
File is unlocked
File is locked
File is unlocked
File is locked
File is unlocked
File is locked
File is unlocked
File is locked
```

Рис. 2: результаты кода

3.2 Реализовать команду man с помощью командного файла

Я изучила содержимое каталога `/usr/share/man/man1`. В нем находятся архивы текстовых файлов, содержащих справку по большинству установленных в системе программ и команд:

```

modulemd-validator.1.gz      umask.1.gz
moggssplit.1.gz             unalias.1.gz
mogriify.1.gz               uname.1.gz
mokutil.1.gz                unexpand.1.gz
montage.1.gz                unflatten.1.gz
more.1.gz                   unicode_start.1.gz
mount.composefs.1.gz        unicode_stop.1.gz
mount.ddi.1.gz              uniq.1.gz
mountpoint.1.gz             unix2dos.1.gz
mpage.1.gz                  unix2mac.1.gz
mpartition.1.gz             unlink.1.gz
mpost.1.gz                  unoconv.1.gz
mptopdf.1.gz                unopkg.1.gz
mpv.1.gz                    unset.1.gz
mrd.1.gz                    unsetfiles.1.gz
mren.1.gz                   unshare.1.gz
msgattrib.1.gz              unsquashfs.1.gz
msgcat.1.gz                 unwrapdiff.1.gz
msgcmp.1.gz                 unxz.1.gz
msgcomm.1.gz                unzip.1.gz
msgconv.1.gz                unzipsfx.1.gz
msgen.1.gz                  unzstd.1.gz
msgexec.1.gz                upbibtex.1.gz
msgfilter.1.gz              updatectl.1.gz
msgfmt.1.gz                 update-desktop-database.1.gz

```

Рис. 3: `ls /usr/share/man/man1`

Потом я создала файл и в нем написала скрипт реализующий команды `man`. Он принимает аргумент `$1`, проверяет существование файла в `/usr/share/man/man1`, и если файл существует, использует `less` для отображения содержимого сжатой страницы руководства. Если файл не существует, выводит "invalid command":



```

1 a=$1
2 if test -f "/usr/share/man/man1/$a.1.gz"
3 then less /usr/share/man/man1/$a.1.gz
4 else
5 echo "Invalid command"
6 fi

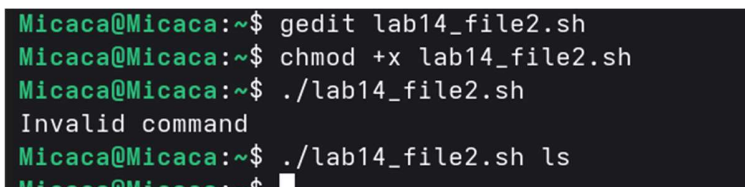
```

Рис. 4: командный файл `man`

```

a=$1
if test -f "/usr/share/man/man1/$a.1.gz"
then less /usr/share/man/man1/$a.1.gz
else
echo "Invalid command"
fi

```



```

Micaca@Micaca:~$ gedit lab14_file2.sh
Micaca@Micaca:~$ chmod +x lab14_file2.sh
Micaca@Micaca:~$ ./lab14_file2.sh
Invalid command
Micaca@Micaca:~$ ./lab14_file2.sh ls
Micaca@Micaca:~$

```

Рис. 5: проверка командного файла `man`

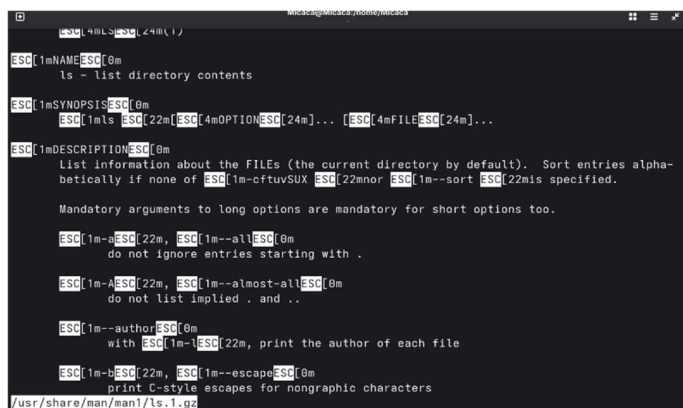


Рис. 6: проверка командного файла `man`

3.3 написать командный файл, генерирующий случайную последовательность букв латинского алфавита.

Я написала скрипт который генерирует случайное число используя `$RANDOM`, а затем с помощью `tr` заменяет каждую цифру на букву от 'a-z' и 'A-Z':



Рис. 7: командный файл, генерирующий случайную последовательность букв
`echo $RANDOM | tr '0-9' 'a-zA-Z'`

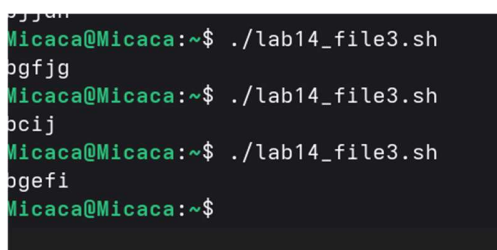


Рис. 8: запуск скрипта

4 Выводы

При выполнении данной работы я научилась писать более сложные командные файлы с использованием логических управляющих конструкций и циклов.

5 Ответы на контрольные вопросы

1. В данной строчке допущены следующие ошибки: не хватает пробелов после первой скобки [и перед второй скобкой] выражение `$1` необходимо взять в "", потому что эта переменная может содержать пробелы. Таким образом, правильный вариант должен выглядеть так: `while [ "$1" != "exit" ]`

2. Чтобы объединить несколько строк в одну, можно воспользоваться несколькими способами: Первый: VAR1="Hello," VAR2=" World" VAR3="VAR1VAR2" echo "\$VAR3"
Результат: Hello, World Второй: VAR1="Hello, " VAR1+=" World" echo "\$VAR1"
Результат: Hello, World
3. Команда seq в Linux используется для генерации чисел от ПЕРВОГО до ПОСЛЕДНЕГО шага INCREMENT. Параметры: seq LAST: если задан только один аргумент, он создает числа от 1 до LAST с шагом шага, равным 1. Если LAST меньше 1, значение не выдает. seq FIRST LAST: когда заданы два аргумента, он генерирует числа от FIRST до LAST с шагом 1, равным 1. Если LAST меньше FIRST, он не выдает никаких выходных данных. seq FIRST INCREMENT LAST: когда заданы три аргумента, он генерирует числа от FIRST до LAST на шаге INCREMENT. Если LAST меньше, чем FIRST, он не производит вывод. seq -f «FORMAT» FIRST INCREMENT LAST: эта команда используется для генерации последовательности в форматированном виде. FIRST и INCREMENT являются необязательными. seq -s «STRING» ПЕРВЫЙ ВКЛЮЧЕНО: Эта команда используется для STRING для разделения чисел. По умолчанию это значение равно /n. FIRST и INCREMENT являются необязательными. seq -w FIRST INCREMENT LAST: эта команда используется для выравнивания ширины путем заполнения начальными нулями. FIRST и INCREMENT являются необязательными.
4. Результатом данного выражения $\$((10/3))$ будет 3, потому что это целочисленное деление без остатка.
5. Отличия командной оболочки zsh от bash: В zsh более быстрое автодополнение для cd с помощью Tab В zsh существует калькулятор zcalc, способный выполнять вычисления внутри терминала В zsh поддерживаются числа с плавающей запятой В zsh поддерживаются структуры данных «хэш» В zsh поддерживается раскрытие полного пути на основе неполных данных В zsh поддерживается замена части пути В zsh есть возможность отображать разделенный экран, такой же как разделенный экран vim
6. for ((a=1; a <= LIMIT; a++)) синтаксис данной конструкции верен, потому что, используя двойные круглые скобки, можно не писать \$ перед переменными ().
7. Преимущества и недостатки скриптового языка bash:

Один из самых распространенных и ставится по умолчанию в большинстве дистрибутивах Linux, MacOS Удобное перенаправление ввода/вывода Большое количество команд для работы с файловыми системами Linux Можно писать собственные скрипты, упрощающие работу в Linux Недостатки скриптового языка bash: Дополнительные библиотеки других языков позволяют выполнить больше действий Bash не является языком общего назначения Утилиты, при выполнении скрипта, запускают свои процессы, которые, в свою очередь, отражаются на скорости выполнения этого скрипта Скрипты, написанные на bash, нельзя запустить на других операционных системах без дополнительных действий

Список литературы

Архитектура ЭВМ